

JWT Microservices Authentication System - 40 Point Submission

Punuar nga: Dion Salihu **Projekti:** Sistem mikrosërbimesh me autentifikim JWT dhe modul AI për trajnim anomalish **Qëllimi i dorëzimit:** 40 pikë nga 100

Scope i Projektit

Ky projekt synon **40 pikë nga 100** duke përfunduar dhe demonstruar këto kërkesa:

Pikët	Kërkesa	Statusi
10	Auth Service me <code>/register</code> , <code>/login</code> , bcrypt dhe JWT HMAC-SHA256 me expiry 30 min	E përfunduar
10	Dy Resource Services që validojnë JWT me sekret të ndarë	E përfunduar
10	AI Module Training me Isolation Forest dhe ruajtje të modelit	E përfunduar
10	Përgjigje me shkrim për konceptet kryesore	E përfunduar

Ky version fokusohet vetëm në këto 4 kërkesa. Kërkesat si TLS/Wireshark, live AI blocking, brute-force simulator, impossible travel real-time, dashboard real-time me SignalR dhe false-positive tuning nuk janë pjesë e këtij dorëzimi 40 pikësh.

Arkitektura e Projektit

Projekti është i ndarë në disa pjesë:

Pjesa	Teknologjia	Roli
<code>auth-service</code>	ASP.NET Core	Regjistron përdorues, verifikon login-in, gjeneron JWT
<code>resource-service-1</code>	ASP.NET Core	API e mbrojtur me JWT
<code>resource-service-2</code>	ASP.NET Core	API e dytë e mbrojtur me JWT
<code>dashboard</code>	Blazor Server	UI për demonstrim të login-it dhe testimit të API-ve
<code>ai-service/train.py</code>	Python, scikit-learn	Trajnon modelin Isolation Forest
<code>postgres-db</code>	PostgreSQL në Docker	Ruan përdoruesit

Rrjedha kryesore:

Përdoruesi -> Dashboard -> Auth Service -> PostgreSQL

Përdoruesi -> Dashboard -> Resource Service 1 / 2 me JWT

AI Module -> `train.py` -> `models/isolation_forest.pkl`

Pas login-it të suksesshëm, `auth-service` kthen një JWT token. Ky token ruhet përkohësisht në sesionin e dashboard-it dhe dërgohet te Resource Services në header:

Authorization: Bearer <JWT_TOKEN>

1. Auth Service - 10 pikë

Çfarë kërkohet

Kërkesa ishte të ndërtohet një Auth Service në ASP.NET Core me:

- endpoint `/register`
- endpoint `/login`
- ruajtje të password-it me bcrypt, jo plaintext
- gjenerim të JWT me HMAC-SHA256
- token me skadim pas 30 minutash

Si është implementuar

Auth Service gjendet në folderin:

`auth-service/`

File kryesor:

auth-service/Controllers/AuthController.cs

Register

Endpoint-i:

POST /api/auth/register

Në register kontrollohet nëse email-i ekziston. Nëse nuk ekziston, krijohet përdoruesi i ri dhe password-i ruhet si hash me bcrypt.

Kodi kryesor:

```
PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password)
```

Kjo do të thotë që password-i real nuk ruhet në databazë. Në databazë ruhet vetëm vlera hash.

Login

Endpoint-i:

POST /api/auth/login

Në login, sistemi gjen përdoruesin sipas email-it dhe verifikon password-in me bcrypt:

```
BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash)
```

Nëse password-i është i saktë, thirret funksioni:

```
GenerateJwtToken(user)
```

Gjenerimi i JWT

JWT gjenerohet në `GenerateJwtToken`.

Token-i përmban claims:

```
new Claim(ClaimTypes.NameIdentifier, user.Id.ToString())  
new Claim(ClaimTypes.Email, user.Email)
```

Këto claims identifikojnë përdoruesin. Token-i nënshkruhet me HMAC-SHA256:

```
new SigningCredentials(key, SecurityAlgorithms.HmacSha256)
```

Token-i skadon pas 30 minutash:

```
expires: DateTime.UtcNow.AddMinutes(30)
```

Konfigurimi i JWT gjendet në:

auth-service/appsettings.json

```
"Jwt": {
  "Key": "THIS_IS_A_SUPER_SECRET_KEY_123456789",
  "Issuer": "auth-service",
  "Audience": "auth-service-users"
}
```

File kryesore në kod

File	Roli
auth-service/Program.cs	Konfigurimi i ASP.NET Core, PostgreSQL, CORS dhe Swagger
auth-service/Controllers/AuthController.cs	Regjistro, login dhe gjenerim JWT
auth-service/Models/User.cs	Modeli i përdoruesit në databazë
auth-service/Data/ApplicationDbContext.cs	Lehtëson PostgreSQL përmes Entity Framework Core
auth-service/appsettings.json	Connection string dhe JWT settings

Dëshmi / Screenshot

Vendos screenshot-et këtu:

Screenshot 1.1 - Kodi i bcrypt në AuthController.cs

Vendos screenshot të rreshtave ku shihet BCrypt.HashPassword dhe BCrypt.Verify.

Screenshot 1.2 - Kodi i JWT në AuthController.cs

Vendos screenshot ku shihet HmacSha256 dhe AddMinutes(30).

Screenshot 1.3 - UI Login/Register

Vendos screenshot nga <http://localhost:5080/login> pas regjistrimit dhe login-it.

Screenshot 1.4 - JWT i kthyer në UI

Vendos screenshot ku shfaqet token-i në dashboard.

Përfundim

Auth Service e plotëson kërkesën sepse mundëson regjistrimin e përdoruesit, ruan password-in me bcrypt, verifikon login-in dhe gjeneron JWT të nënshkruar me HMAC-SHA256 me afat 30 minuta.

2. Resource Services - 10 pikë

Çfarë kërkohej

Kërkesa ishte të krijohen dy API Resource Services që validojnë JWT me sekret të përbashkët dhe të demonstrohet:

- pa token -> 401 Unauthorized
- me token valid -> 200 OK
- me token të pavlefshëm/të ndryshuar -> 401 Unauthorized

Si është implementuar

Resource Services janë:

`resource-service-1/`
`resource-service-2/`

Secili service ka endpoint publik:

`GET /public`

dhe endpoint të mbrojtur:

`GET /secure`

Endpoint-i `/secure` kërkon autentifikim:

```
}).RequireAuthorization();
```

Si validohet JWT

Në `Program.cs` të secilit Resource Service është konfiguruar JWT Bearer Authentication:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true
        };
    });
```

Këto kontrolle nënkuptojnë:

Kontrolli	Kuptimi
ValidateIssuer	Kontrollon nëse token-i është lëshuar nga auth-service
ValidateAudience	Kontrollon audience-in e token-it
ValidateLifetime	Kontrollon nëse token-i ka skaduar
ValidateIssuerSigningKey	Kontrollon nënshkrimin me sekretin e përbashkët

Resource Services përdorin të njëjtin konfigurim JWT si Auth Service:

```
"Jwt": {
  "Key": "THIS_IS_A_SUPER_SECRET_KEY_123456789",
  "Issuer": "auth-service",
  "Audience": "auth-service-users"
}
```

Kjo është arsyeja pse ato mund ta validojnë token-in pa pyetur Auth Service për çdo request.

File kryesore në kod

File	Roli
resource-service-1/Program.cs	JWT validation dhe endpoint-et /public, /secure
resource-service-2/Program.cs	JWT validation dhe endpoint-et /public, /secure
resource-service-1/appsettings.json	JWT settings për service 1
resource-service-2/appsettings.json	JWT settings për service 2
dashboard/Pages/ApiTest.razor	UI për testimin e 401/200/401

Dëshmi / Screenshot

Vendos screenshot-et këtu:

Screenshot 2.1 - Kodi i JWT validation në Resource Service

Vendos screenshot ku shihet AddJwtBearer, ValidateIssuer, ValidateAudience, ValidateLifetime dhe ValidateIssuerSigningKey.

Screenshot 2.2 - Kodi i endpoint-it të mbrojtur

Vendos screenshot ku shihet /secure dhe RequireAuthorization().

Screenshot 2.3 - UI pa token -> 401

Vendos screenshot nga dashboard ku Resource 1 ose 2 pa token kthen HTTP 401.

Screenshot 2.4 - UI me JWT valid -> 200

Vendos screenshot ku Resource 1 ose 2 me token valid kthen HTTP 200.

Screenshot 2.5 - UI me token të pavlefshëm -> 401

Vendos screenshot ku token i ndryshuar/tampered kthen HTTP 401.

Përfundim

Dy Resource Services e plotësojnë kërkesën sepse secili e validon JWT token-in vetë. Nëse token-i mungon ose është i pavlefshëm, përgjigjja është 401. Nëse token-i është valid, endpoint-i /secure kthen 200.

3. AI Module Training - 10 pikë

Çfarë kërkohej

Kërkesa ishte të trajnohet një model AI me Isolation Forest mbi të dhëna login-i dhe të ruhet modeli.

Në këtë projekt, AI Module fokusohet te pjesa e trajnimit:

`ai-service/train.py`

Isolation Forest

Isolation Forest është algoritëm unsupervised për anomaly detection. Ai nuk kërkon që çdo shembull të jetë i etiketuar si sulm ose jo-sulm. Ideja është që pikat e pazakonta izolohen më shpejt në pemët rastësore, prandaj mund të identifikohen si anomali.

Ky algoritëm është i përshtatshëm për login security sepse në praktikë është më e lehtë të mbledhim sjellje normale login-i sesa të kemi shumë mostra reale sulmesh.

Si është implementuar trajnimi

Në `train.py` krijohen mostra sintetike login-i:

- 180 login normale
- 40 login normale me variacione të lehta
- 15 raste brute-force / credential stuffing
- 15 raste impossible travel

Totali është:

250 mostra trajnimi

Pastaj të dhënat vendosen në një `DataFrame` të `pandas`:

```
train_df = pd.DataFrame(samples, columns=columns)
```

Modeli krijohet me `scikit-learn`:

```
model = IsolationForest(  
    contamination=0.08,  
    random_state=42  
)
```

Trajnimi bëhet me:

```
model.fit(train_df)
```

Features e login-it

Modeli trajnohet me këto features:

Feature	Kuptimi
hour	Ora kur ndodh login-i
failed_attempts	Numri i tentimeve të dështuara
new_ip	A është IP e re për përdoruesin
success_rate	Norma e suksesit të login-it
login_frequency	Sa shpesh po bëhet login
country_changed	A ka ndryshuar shteti
impossible_travel	A duket udhëtim i pamundur
distance_km	Distanca ndërmjet login-eve
hours_since_last_login	Sa orë kanë kaluar nga login-i i fundit
user_agent_changed	A ka ndryshuar browser/device

Ruajtja e modelit

Modeli i trajnuar ruhet në:

```
models/isolation_forest.pkl
```

Kodi:

```
joblib.dump(model, MODELS_DIR / "isolation_forest.pkl")
```

Gjithashtu ruhet konfigurimi:

```
models/model_config.json
```

me threshold:

```
{  
    "threshold": -0.1  
}
```


Komanda për trajnim

Nga folderi kryesor i projektit:

```
ai-service/.venv/bin/python ai-service/train.py
```

Output-i i pritur:

```
Model trained and saved successfully!  
Threshold saved to models/model_config.json
```

File kryesore në kod

File	Roli
ai-service/train.py	Krijon dataset-in, trajnon Isolation Forest dhe ruan modelin
ai-service/requirements.txt	Libraritë Python të nevojshme
models/isolation_forest.pkl	Modeli i trajnuar
models/model_config.json	Konfigurimi i threshold-it
dashboard/Pages/AiModel.razor	UI që tregon statusin e modelit

Dëshmi / Screenshot

Vendos screenshot-et këtu:

Screenshot 3.1 - Kodi i features në train.py

Vendos screenshot ku shihet lista columns me features.

Screenshot 3.2 - Kodi i IsolationForest

Vendos screenshot ku shihet IsolationForest, model.fit dhe joblib.dump.

Screenshot 3.3 - Terminali pas ekzekutimit të train.py

Vendos screenshot ku shihet output:
Model trained and saved successfully!

Screenshot 3.4 - Folderi models/

Vendos screenshot ku shihen:
models/isolation_forest.pkl
models/model_config.json

Përfundim

AI Module e plotëson kërkesën sepse trajnon një model Isolation Forest mbi features të login-it dhe e ruan modelin në file .pkl, së bashku me konfigurimin e threshold-it.

4. Përgjigjet me shkrim - 10 pikë

Çfarë kërkohej

Kërkohej një dokument me përgjigje teorike për këto tema:

1. JWT vs Session
2. bcrypt vs SHA-256
3. Isolation Forest
4. Pse rate limiting nuk mjafton
5. Krahësim me Auth0 / Keycloak / Cognito

Përgjigjet janë të lidhura me mënyrën si është ndërtuar projekti.

(a) JWT vs Session

Session-based authentication ruan gjendjen e përdoruesit në server. Kur përdoruesi bën login, serveri krijon një session ID dhe zakonisht e dërgon te klienti në cookie. Në çdo request të ri, serveri duhet ta kërkojë atë session në memorie, cache ose databazë.

Kjo qasje është e thjeshtë për aplikacione monolitike, por bëhet më e vështirë në mikrosërbbime. Nëse kemi disa services, atëherë të gjitha duhet të kenë qasje te i njëjti session storage ose duhet të përdoret sticky session. Kjo e rrit kompleksitetin.

JWT authentication është stateless. Pas login-it, Auth Service krijon një token të nënshkruar dhe ia kthen klientit. Klienti e dërgon token-in në çdo request:

Authorization: Bearer <JWT_TOKEN>

Resource Services e validojnë token-in vetë duke kontrolluar nënshkrimin, issuer, audience dhe expiry. Kjo është e përshtatshme për mikrosërbbime sepse nuk ka nevojë që çdo service ta pyesë Auth Service për çdo request.

Në këtë projekt, **auth-service** lëshon JWT, ndërsa **resource-service-1** dhe **resource-service-2** e validojnë token-in me të njëjtin sekret.

(b) bcrypt vs SHA-256

SHA-256 është algoritëm hash shumë i shpejtë. Ai është i mirë për integritetin e të dhënave, për shembull për të kontrolluar nëse një file është ndryshuar. Por për password-e nuk është zgjedhje e mirë, sepse shpejtësia e tij i ndihmon sulmuesit të provojnë shumë kombinime në kohë të shkurtër.

bcrypt është projektuar posaçërisht për password-e. Ai përdor salt unik dhe ka work factor, që e bën procesin më të ngadalshëm dhe më të vështirë për brute-force offline.

Në këtë projekt, password-i ruhet kështu:

```
BCrypt.Net.BCrypt.HashPassword(dto.Password)
```

dhe verifikohet kështu:

```
BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash)
```

Kjo do të thotë që password-i real nuk ruhet në databazë. Në databazë ruhet vetëm hash-i.

(c) Isolation Forest

Isolation Forest është algoritëm për anomaly detection. Ai funksionon duke krijuar pemë rastësore dhe duke matur sa shpejt izolohet një pikë. Pikat normale zakonisht kërkojnë më shumë ndarje për t'u izoluar, ndërsa pikat e pazakonta izolohen më shpejt.

Ky algoritëm është i dobishëm për login security sepse sjelljet e dyshimta shpesh dallohen nga sjellja normale:

- shumë tentativa të dështuara
- login në orare të pazakonta
- IP e re
- ndryshim shteti
- distancë shumë e madhe ndërmjet dy login-eve
- ndryshim i browser-it ose pajisjes

Në këtë projekt, `ai-service/train.py` krijon dataset me login normale dhe disa raste anomali, trajnon modelin `IsolationForest` dhe e ruan modelin në `models/isolation_forest.pkl`.

(d) Pse rate limiting nuk mjafton

Rate limiting kontrollon vetëm shpejtësinë ose numrin e request-ve. Për shembull, mund të kufizojë sa login attempts lejohen brenda një minute.

Kjo ndihmon kundër sulmeve të thjeshta brute-force, por nuk mjafton për raste më të avancuara:

- credential stuffing mund të bëhet ngadalë, pa kaluar limitin
- login nga një lokacion i pamundur nuk zbulohet vetëm nga numri i request-ve

- një login në orë të pazakontë mund të jetë i dyshimtë edhe nëse është vetëm një request
- ndryshimi i browser-it, IP-së dhe vendit kërkon analizë konteksti

Prandaj AI anomaly detection mund të analizojë më shumë features së bashku, jo vetëm numrin e request-ve. Në këtë projekt, modeli trajnohet për të kuptuar dallimin mes sjelljes normale dhe sjelljes së pazakontë.

(e) Krahësim me Auth0 / Keycloak / Cognito

Platforma	Përshkrimi	Përparësitë	Kufizimet
Auth0	SaaS identity provider	Setup i shpejtë, OAuth/OIDC, dashboard i gatshëm	Varësi nga palë e tretë, kosto
Keycloak	Open-source identity provider	Kontroll i lartë, self-hosted, enterprise features	Setup dhe mirëmbajtje më komplekse
AWS Cognito	Shërbim i AWS për auth	Integrim i mirë me AWS, menaxhim cloud	I lidhur me ekosistemin AWS
Ky projekt	Implementim edukativ custom	Tregon JWT, bcrypt, mikrohërbime dhe AI training në kod	Jo production-ready pa hardening shtesë

Ky projekt nuk synon të zëvendësojë Auth0, Keycloak ose Cognito në prodhim. Qëllimi është edukativ: të demonstrohet si funksionon autentifikimi JWT në mikrohërbime dhe si mund të trajnohet një model AI për login anomaly detection.

Dëshmi / Screenshot

Vendos screenshot-et këtu:

Screenshot 4.1 - Dokumenti me përgjigje teorike

Vendos screenshot nga ky seksion ose nga PDF final.

Screenshot 4.2 - PDF/Word final

Vendos screenshot ku shihet dokumenti i eksportuar.

Përfundim

Kërkesa e përgjigjeve me shkrim plotësohet sepse dokumenti shpjegon konceptet kryesore të projektit: JWT, sessions, bcrypt, SHA-256, Isolation Forest, rate limiting dhe krahasimin me platforma ekzistuese të autentifikimit.

Komandat për Demo dhe Testim

Nisja e projektit

Nga folderi kryesor:

```
docker compose up -d
dotnet ef database update --project auth-service/auth-service.csproj
./scripts/run-all-services.sh
```

Pas nisjes, hapet:

Dashboard: <http://localhost:5080>

Auth Swagger: <http://localhost:5038/swagger>

Testimi automatik

```
python3 scripts/run-integration-tests.py
```

Output-i i pritur:

```
=== Results: 12 passed, 0 failed ===
```

Trajnimi i AI modelit

```
ai-service/.venv/bin/python ai-service/train.py
```

Output-i i pritur:

```
Model trained and saved successfully!
```

```
Threshold saved to models/model_config.json
```

Checklist për Screenshot

Nr.	Screenshot	Ku merret
1	Scope 40 pikë	README ose ky dokument
2	Register/Login UI	http://localhost:5080/login

Nr.	Screenshot	Ku merret
3	JWT i shfaqur	<code>http://localhost:5080/login</code> pas login-it
4	bcrypt në kod	<code>auth-service/Controllers/AuthController.cs</code>
5	JWT HMAC-SHA256 + expiry 30 min	<code>auth-service/Controllers/AuthController.cs</code>
6	JWT validation	<code>resource-service-1/Program.cs</code>
7	<code>/secure</code> me <code>RequireAuthorization</code> ose	<code>resource-service-1/Program.cs</code> <code>resource-service-2/Program.cs</code>
8	401 pa token	<code>http://localhost:5080/api-test</code>
9	200 me token valid	<code>http://localhost:5080/api-test</code>
10	401 me token të pavlefshëm	<code>http://localhost:5080/api-test</code>
11	Features në <code>train.py</code>	<code>ai-service/train.py</code>
12	IsolationForest + <code>joblib.dump</code>	<code>ai-service/train.py</code>
13	Terminali pas <code>train.py</code>	Terminal
14	<code>models/isolation_forest.pkl</code> dhe <code>model_config.json</code>	File explorer ose terminal
15	12 passed, 0 failed	Terminal pas integration tests

Përfundim i Përgjithshëm

Ky dorëzim demonstroi 4 kërkesa kryesore për 40 pikë:

1. Auth Service regjistron përdorues, ruan password-in me bcrypt dhe gjeneron JWT 30-minutësh.
2. Dy Resource Services validojnë JWT me sekret të përbashkët dhe mbrojnë endpoint-in `/secure`.
3. AI Module trajnon modelin Isolation Forest mbi features të login-it dhe ruan modelin.
4. Dokumenti shpjegon konceptet kryesore teorike dhe lidhjen e tyre me implementimin.

Me këtë strukturë, projekti demonstroi qartë autentifikimin stateless me JWT në një arkitekturë mikrosërvisesh dhe pjesën bazë të AI training për login anomaly detection.